

W-CTC: A Connectionist Temporal Classification Loss with Wild Cards

Saksham Katiyar, Chinmay Pimpalkhare, Vikhyat Agrawal, Ankur Verma

Indian Institute of Technology Bombay

CS 753: Automatic Speech Recognition

Outline

- 1 Introduction
- 2 Wild-Card CTC Algorithm
- 3 Experiments
- 4 Appendices

Outline

- 1 Introduction
- 2 Wild-Card CTC Algorithm
- 3 Experiments
- 4 Appendices

- Connectionist Temporal Classification (CTC) is used for sequence to sequence learning applications
- Standard CTC computes a loss by aggregating over all possible alignment paths, that map the entire input sequence to the entire label (full alignment)
- This paper proposes Wildcard-CTC (W-CTC) to address the issue of partial alignment, particularly when the labels match only the middle part of the input sequence

Some examples where the labels are partially aligned with the input sequence:

- A speech transcription often omits the prefacing introduction of the speaker
- A beginning of an audio book that tells who is reading the book, along with various other details, are typically not in the written version
- A sign language interpreter often ignore the irrelevant gestures in the beginning and ending stages
- The label is corrupted or noisy due to variable recording conditions.

Problem Formulation

Let $X = [X_0, X_1, \dots, X_{T-1}]$ be the input sequence of length T . These could be:

- Speech audio frames for ASR System
- Image segments for OCR System

Let $Y = [Y_0, Y_1, \dots, Y_{N-1}]$ be the corresponding targets (label) of length N . These could be:

- Transcription for ASR System
- Recognized string for OCR System

Assume $T \geq N$

Problem Formulation

Let the alignment function be $\psi : X \rightarrow Y$, which maps a frame $x \in X$ to a target $y \in Y$

Definition 1 (Subsequence). A **subsequence** of X , denoted as $\hat{X} \subset X$, is a vector of multiple **consecutive** elements in X , i.e.

$\hat{X} = [X_i, X_{i+1}, \dots, X_{i+\tau}]$, where $\tau > 0$.

Definition 2 (Full alignment). A **full** alignment maps the entire X to the entire Y , i.e. $y = \psi(x) \in Y$, for any $x \in X$, as well as $x = \psi^{-1}(y) \in X$, for any $y \in Y$.

Definition 3 (Alignment path). An alignment **path**, π , is a vector of the same length as X , i.e. $\pi = [y_0, y_1, \dots, y_{T-1}]$, where each element $y_i = \psi(X_i) \in Y$, for $0 \leq i < T$.

Definition 4 (Valid alignment). An alignment path is **valid**, if it satisfies:

- Monotonicity: If X_i maps to Y_j , X_{i+t} cannot map to Y_{j-k} , for any $t > 0$ and $k > 0$.
- Surjection (many-to-one mapping): $\psi(.)$ is a surjective function, such that a subsequence $\hat{X} \subset X$, could map to a single $Y_j \in Y$. But each X_i can have only one label $y \in Y$.

Definition 5 (Partial Alignment Problem). A problem where the label is a subsequence $\hat{Y} = [Y_k, Y_{k+1}, \dots, Y_{k+L-1}] \subset Y$, that has a length $L < N$. Both k and L are **unknown**.

Brief Introduction of CTC

Consider the **full alignment problem**. There could be many paths leading to the same label Y from input X . Let $\mathcal{Z}$ be the set of all valid paths. The standard CTC loss is defined as the negative log-likelihood of the labels given input sequences, i.e.

$$\mathcal{L}_{CTC}(X, Y) = -\log P(Y|X) = -\log \sum_{\pi \in \mathcal{Z}} P_{\pi}(Y|X)$$

where $P_{\pi}(Y|X)$ is the probability of a particular alignment path π . For any path, CTC assumes conditional independence amongst X_i 's, $0 \leq i < T$. So the probability is approximated as:

$$P_{\pi}(Y|X) \approx \prod_{0 \leq t < T} P_t(\pi[t] | X)$$

Brief Introduction of CTC

This paper reduces the **partial alignment problem** to the **full alignment** problem, by trying to do full alignment between the corrupted targets, $\hat{Y}$, and all possible subsequences of X .

Let $\hat{X}_{i,\tau} = [X_i, X_{i+1}, \dots, X_{i+\tau}]$ be one subsequence, starting from i -th frame with a length of $\tau + 1$. One way is to loop over all possible (i, τ) pairs, obtain the CTC loss for each subsequence, and summarize them, i.e.

$$\mathcal{L}'_{CTC}(X, \hat{Y}) = \sigma\{\mathcal{L}_{CTC}(\hat{X}_{i,\tau}, \hat{Y}) \mid i + \tau < T\}$$

where $\sigma(\cdot)$ means a summary of the set, e.g. summation or max.

Time Complexity: $O(T^3N)$

If we reuse the DP trellis for a fixed i then time complexity reduces to $O(T^2N)$ which is still hardly deployable in practice since T can be fairly large.

Outline

- 1 Introduction
- 2 Wild-Card CTC Algorithm
- 3 Experiments
- 4 Appendices

Quick Recap: CTC Algorithm

Summary of the steps taken to compute CTC:

- Usage of blank symbols "-" (or ϵ) in between 2 consecutive characters (Making length of the transcription from N to $2N + 1$)
- Build a trellis matrix containing $2N + 1$ rows and T columns
- We take the sum of recurrent terms accumulated at the last column for rows $2N$ and $2N + 1$ to be the CTC loss

Quick Recap: CTC Algorithm

Formally, we can write the CTC loss as

$$\mathcal{L}_{CTC}(X, \tilde{Y}) = -\log(M_{2N-1, T-1} + M_{2N, T-1})$$

where $M_{i,j} = P(\tilde{Y}[0 : i] | X[0 : j])$, which can be written as a recurrence relation as

$$M_{i,j} = \begin{cases} P(\tilde{Y}_i | X)(M_{i,j-1} + M_{i-1,j-1}), & \text{if } \tilde{Y}_i = "-" \text{ or } \tilde{Y}_i = \tilde{Y}_{i-2} \\ P(\tilde{Y}_i | X)(M_{i,j-1} + M_{i-1,j-1} + M_{i-2,j-1}), & \text{otherwise} \end{cases}$$

Proposed W-CTC Algorithm

Building on the original CTC algorithm, the W-CTC algorithm follows as:

- In addition to the blank symbol, we now insert a wild card symbol "*" at the start of every alignment. For example, "hello" would be converted to "*-h-e-l-l-o-"
- The wild card symbol can be used to represent any symbol in the vocabulary, including the blank symbol.
- The new alignment has a length of $2N + 2$ and the trellis has $2N + 2$ rows

Proposed W-CTC Algorithm

- Instead of 2 initial states in the trellis (blank and first character), we also include the wild card symbol as a possible start state, leading to a total of 3 starting nodes.
- Here, $M_{0,0} = P(*|X_0) = 1$

Proposed W-CTC Algorithm

- Instead of inserting a second wild card symbol at the end of the alignment, we change the computation of the CTC loss
- Instead of taking only the final states $M_{2N-1,T-1}, M_{2N,T-1}$ to compute the CTC loss, we consider the entire last 2 rows
- Formally, compute

$$\mathcal{L}_{CTC}^{(j)} = -\log(M_{2N,j} + M_{2N+1,j})$$

for $j = N - 1$ to $T - 1$

- Summarize these column-wise CTC losses into a single loss function (W-CTC) as

$$\mathcal{L}_{W-CTC} = \sigma\{\mathcal{L}_{CTC}^{(j)} | N - 1 \leq j < T\}$$

where σ is the softmax function (reasons given later)

- Note that $j < N - 1$ is not considered because they would not correspond to valid alignments

Visualization of W-CTC

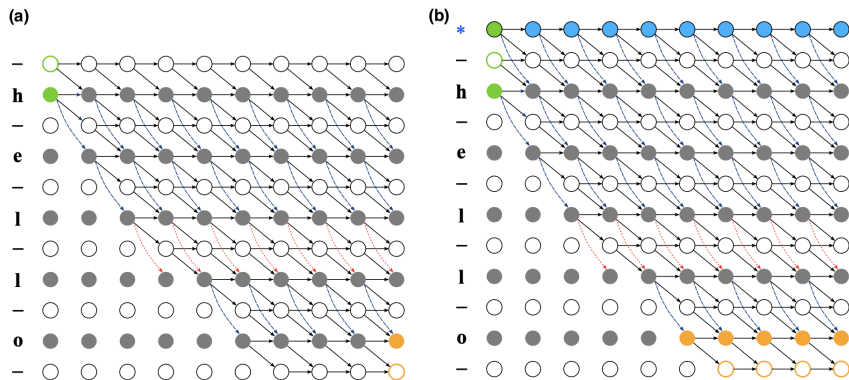


Figure: DP Trellis for (a) Standard CTC, (b) Wild Card CTC (W-CTC)

Things to Note

- $P(*|X_0) = 1$ implies that there will be no punishment for going from $M_{0,j-1}$ to $M_{0,j}$. This means that the start of the transcription can be anywhere in the audio file
- The wild-card symbol "*" is **NOT** the same as the blank symbol "-"
- Standard CTC focusses on the full alignment problem whereas the W-CTC focusses on the partial alignment problem

Complexities of W-CTC

- Time and Space Complexities are same as the original CTC loss because we are just adding another row in the trellis.
- Time Complexity : $\mathcal{O}(TN)$
- Space Complexity : $\mathcal{O}(TN)$

Limitations

- W-CTC only handles missing labels, and not missing inputs
- The missing labels (Y) need to be present at either of the two ends, and not in the middle. For example, "h???o" cannot be handled whereas "?ell?" can be handled
- The algorithm requires the label (Y) to be a continuous subsequence. For example, "?ell?" can be solved, but "?e?lo" cannot.

Outline

- 1 Introduction
- 2 Wild-Card CTC Algorithm
- 3 Experiments**
- 4 Appendices

Experimental settings

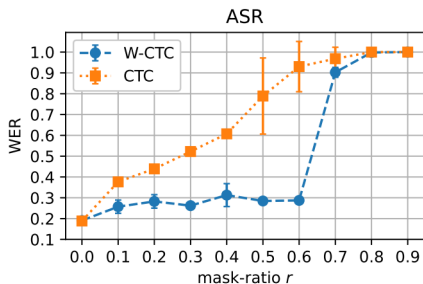
- Experiments on Speech, OCR Sign Language tasks
- Randomly mask out 'r' fraction of each of the training labels ($r \in (0, 0.9)$)
- Test set is left unmasked
- Standard CTC used as baseline

Masking Procedure

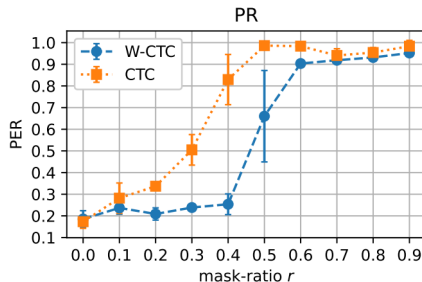
Illustrative Example

- Suppose we have the label string **“helloworld”**, 10 characters in total
- We want to mask 50%, which is 5 characters
- We will only mask front and end, randomly
- Generate a random starting position, uniformly from $[0, 5]$
- Suppose the RNG yields 3, then keep the characters from 3 to 7 and throw out the rest
- The training label in this case would be **“llowo”**

Results on Speech tasks



ASR



Phoneme Recognition

- Above results on TIMIT dataset, which includes audios and corresponding transcriptions at word and phoneme level
- Backbone model is the pretrained Wav2vec-2.0-base

- W-CTC always performs better than CTC
- Only exception at $r = 0$, when CTC performs better than W-CTC, this is because of the wild-card symbol

- The OCR task inputs an image containing text, and outputs recognized text with their location on the image
- Two major components: text detection and text recognition
- CTC widely used in text recognition block
- MJSynth (9 million images) and SynthText (800k images) used for training and then evaluation on 7 standard benchmarks
- Recognition accuracy (ACC), defined as the number of correct images divided by total number of images. Correct images means the predicted text exactly matches the ground truth

OCR task results

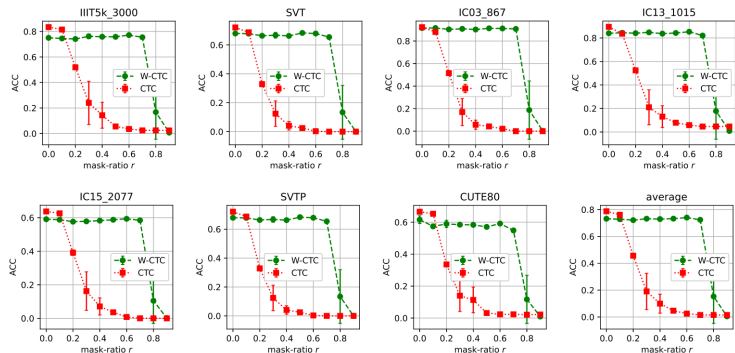


Figure: test accuracy on the 7 standard benchmarks

Demonstration of partial alignment

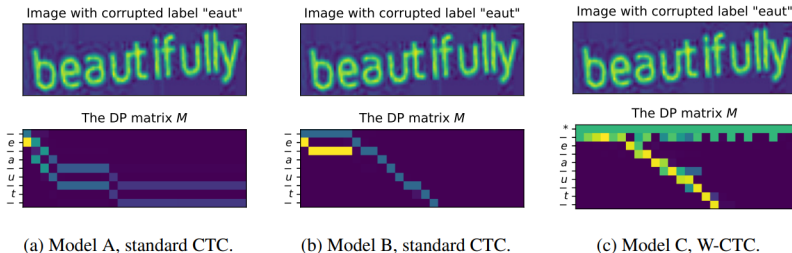


Figure: Model A,C using masked labels, Model B using full labels

Everything other than "beaut" is masked

Sign language task

- CSLR - Continuous Sign Language Recognition, use CTC because no temporal alignment is provided in the data
- PHOENIX14T dataset used, which has both spoken language translations and gloss-level annotations

Table 1: WER on CSLR task. W-CTC significantly outperforms CTC when label corrupted.

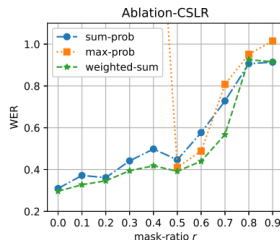
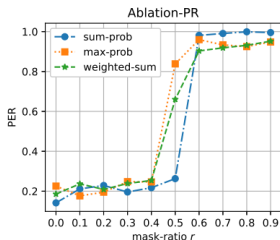
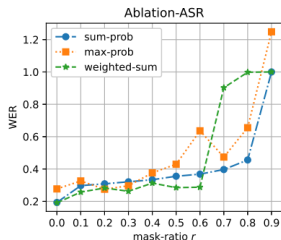
mask ratio r	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9
CTC	0.286	0.422	0.537	0.663	0.735	0.814	0.896	0.945	0.989	0.981
W-CTC	0.297	0.328	0.346	0.395	0.418	0.392	0.440	0.568	0.926	0.916

Ablation study on choice of σ

Few possible choices for σ :-

$$\sigma\{L_{CTC}^{(j)}\} = \begin{cases} \text{sum-prob} : -\log(\sum_j P(Y|X[0:j])) \\ \text{max-prob} : -\log(\max_j P(Y|X[0:j])) \\ \text{weighted-sum} : \textit{softmax} \end{cases}$$

Results for different σ



- Max-prob performs worst for CSLR
- Sum-prob and weighted-sum perform quite similarly, but weighted-sum was chosen due to its lower WER at lower values of 'r'

Outline

- 1 Introduction
- 2 Wild-Card CTC Algorithm
- 3 Experiments
- 4 Appendices**

Hyperparameters

Table 3: List of key training hyper-parameters.

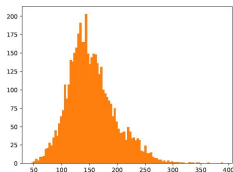
	Batch-size	Optimizer	LR	Steps
ASR	32	AdamW	1e-4	7k (50 epochs)
PR	32	AdamW	1e-4	7k (50 epochs)
OCR	500	AdaDelta	1	150k
CSLR	32	Adam	1e-3	Stop if no better for 800 steps

Figure: List of training hyperparameters used

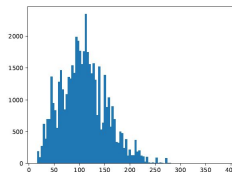
Important Observations

- Learning rate significantly higher for OCR, possibly due to AdaDelta optimizer being used

Sequence Lengths (Durations) and their Effects



(a) TIMIT training set.



(b) PHONEX14T dataset.

	mean	std	min	max
TIMIT	153.60	43.05	46	389
PHONEX14T	108.67	45.25	16	395

- **Figure:** The histogram of duration for TIMIT (for ASR and PR), and PHONEX14T (for CSLR) dataset.
- **Table:** Duration (# of frames) for TIMIT training set (in ASR and PR experiments), and the PHONEX14T dataset (in CSLR experiment).

Sequence Lengths (Durations) and their Effects - II

Insights and Discussion

- For OCR, the image size is fixed to 30×100 , so the duration (proportional to the image length, which is 100), is fixed.
- T varies quite a lot. Should we normalize the sequence lengths?
- **Empirical Observation:** Normalization has little impact on performance.

Experiments on Random Mask-Ratio

- For each utterance, sample mask-ratio r uniformly from between 0 and 1. $\mathbb{E}[r] = 0.5$.

WER	W-CTC			CTC	
	random r	$r = 0.5$	$r = 0$	$r = 0.5$	$r = 0$
mean	0.260	0.285	0.191	0.789	0.189
std	0.013	0.005	0.007	0.183	0.005

Figure: ASR experiments on random-mask ratio (TIMIT Dataset)

Insights

- As expected, random r has results similar to as $r = 0.5$
- For W-CTC, the results are not far from $r = 0$ (clean label case), which shows that W-CTC is indeed effective
- W-CTC results are much better than CTC when $r > 0$

Normalization Effects

- Due to the addition of the wild-card symbol *, with a constant probability 1, the total probability accumulating T frames is no longer normalized.
- To be precise, 2^T is the normalization factor
- To make $\mathbb{P}(Y|X)$ a proper likelihood, we should normalize it as $\mathbb{P}(Y|X)/Z$
- Otherwise, the larger T , the higher the associated values of $\mathbb{P}(Y|X)$ can get
- Due to this, the loss $\mathcal{L}_{W-CTC} = -\log \mathbb{P}(Y|X)$, for longer sequences would be smaller than the shorter ones.
- A **mitigation** is to obtain normalized W-CTC loss:

$$\mathcal{L}_{norm} = -\log \left(\frac{\mathbb{P}(Y|X)}{2^T} \right) \approx \mathcal{L}_{W-CTC} + 0.3T$$

Normalization Effects - II

Normalized loss is given by

$$\mathcal{L}_{norm} = -\log \left(\frac{\mathbb{P}(Y|X)}{2^T} \right) \approx \mathcal{L}_{W-CTC} + 0.3T$$

- The longer the sequence, the more the punishment added to the original loss

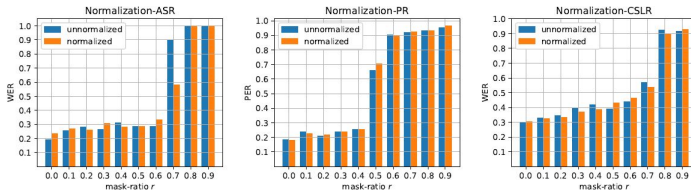


Figure: Comparison of normalized and unnormalized losses. No significant difference or clear trend was observed.

Wild-Card Emission Probability

- The paper sets $\mathbb{P}(\text{wild-card}) = 1$, meaning that it matches any character in the label
- However, trying to make $\mathbb{P}(*|X) = 1$ and $\sum_s \mathbb{P}(s|X) = 1$ (where $s \in \text{Vocab}$ at the same time, leads to a total probability of 2
- The first solution tried out to solve this was normalization
- The other method tried out was to assign $\mathbb{P}(*|X) = p$ and $\sum_s \mathbb{P}(s|X) = 1 - p$
- **Advantage:** We get a normalized probability for each frame
- **Disadvantage 1 :** Need to tune the hyperparameter p
- **Disadvantage 2 :** Slightly alters the meaning of the wild-card *
- **Empirical Observation:** No significant difference observed in WER, hence unnormalized version chosen

Convergence behaviour: Training loss comparison

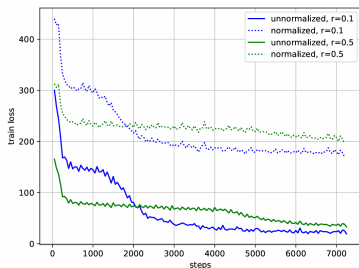


Figure: The training loss comparison between the unnormalized version and the normalized version

Here, the losses are compared for mask ratio $r=0.1$ and 0.5 , for PR task (using TIMIT dataset). The plot shows that the normalization term here does not affect the training trend in general, but simply adding a constant values on top ($+0.3T$).

Discussion on Wild-Card and Weighted Sum

Why do we put a wild-card only in the beginning?

- If the wild-card is also appended at the end, training failure was observed due to improper accumulation of probabilities along paths

Why do we use weighted-sum?

- Taking hard maximum, which is the intuitively correct choice may ignore last few frames if the best path ends early, which may lead to convergence and back-propagation issues. Hence, a soft-max version was used instead.

Why is weighted-sum only done at the end?

- The metric is obtained only at the end, hence hard selection of the starting position is not a good idea. In contrast, a wild-card is both conceptually simpler and easier for implementation.

Illustrating W-CTC with OCR Examples

- We try to illustrate the diff in trellises between W-CTC and CTC
- We choose OCR examples, where the words are long (10+ chars)
- We shall be comparing three models in the next few slides, them being
 - **Left**: Standard CTC trained on corrupted labels with mask-ratio $r = 0.7$
 - **Middle**: Standard CTC trained on clean labels
 - **Right**: W-CTC trained on corrupted labels with mask ratio $r = 0.7$

OCR: Clean Images

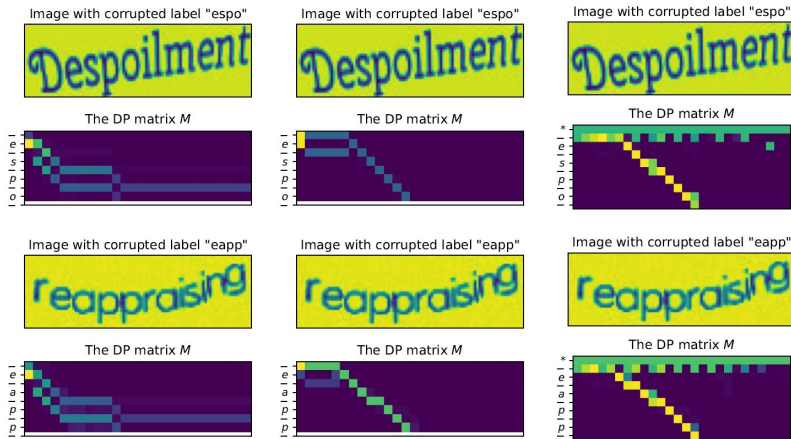


Figure: Standard CTC only makes reasonable inference if it is trained on the clean label. W-CTC generally outputs correct paths

OCR: High Confusion Images

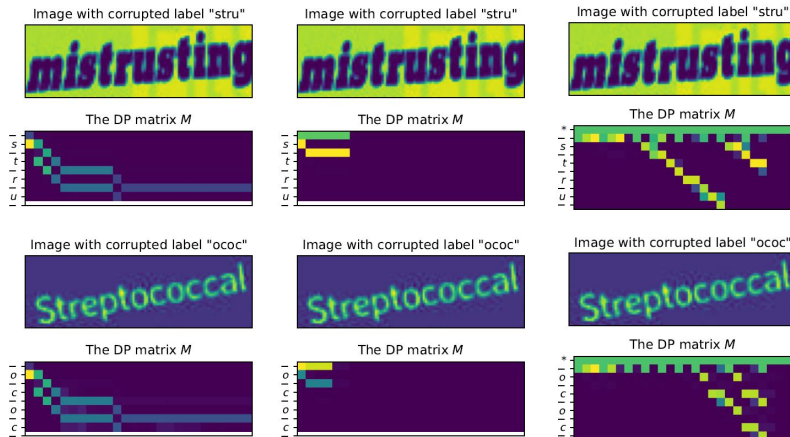


Figure: Standard CTC completely fails. W-CTC succeeds. Repeated character sequences lead to some high confidence paths in W-CTC, but they get cut-off before ending.

Thank You!